

Design and Implementation of a Moore Machine for "11011" Overlapping Sequence Detection

EE24BTECH11024 Abhimanyu Koushik
EE24BTECH11002 Agamjot Singh
IIT Hyderabad

April 21, 2025

1 Aim

To design and implement a Moore machine-based sequence detector that identifies the pattern "11011" with overlapping capability using FSM theory, with sequence generation via Arduino and output visualization through an LED and oscilloscope.

2 Apparatus

- Arduino Uno
- 7474 ICs (D flip-flops) - 2 pieces
- 3-Input AND Gate IC (7441)
- 2-Input OR Gate IC (7432)
- Breadboard and connecting wires
- LED and appropriate resistors ($220\ \Omega$)
- Cathode Ray Oscilloscope (CRO)
- 5V DC power supply

3 Theory

3.1 D Flip-Flops and Sequential Logic

The D flip-flop is a fundamental memory element in sequential logic. Its operation can be described by:

$$Q(t+1) = D(t)$$

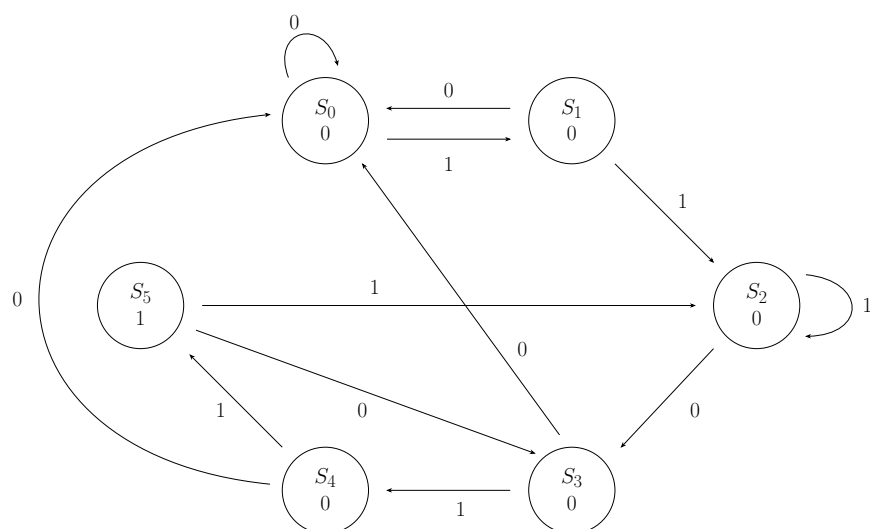
where the next state directly reflects the data input at each clock edge.

3.2 State Diagram

The Moore machine state diagram comprises six states:

- S_0 : Initial state (no bits detected)
- S_1 : Detected “1”
- S_2 : Detected “11”
- S_3 : Detected “110”
- S_4 : Detected “1101”
- S_5 : Detected “11011” (complete sequence)

Transitions between states are triggered by the input bits, and only state S_5 produces an output 1.



3.3 State Encoding

For hardware implementation, the states are encoded using three flip-flops:

S_0	$\rightarrow$	000
S_1	$\rightarrow$	001
S_2	$\rightarrow$	011
S_3	$\rightarrow$	010
S_4	$\rightarrow$	110
S_5	$\rightarrow$	111

3.4 State Transition Table

The table below defines the Moore machine behavior by listing the current state, input, next state, and output. In addition, for hardware implementation, an intermediate transition table includes values for flip-flop inputs.

i/p	Current State			o/p	Next State		
	Q_2	Q_1	Q_0		D_2 or Q'_2	D_1 or Q'_1	D_0 or Q'_0
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0
1	0	0	1	0	0	1	1
0	0	1	1	0	0	1	0
1	0	1	1	0	0	1	1
0	0	1	0	0	0	0	0
1	0	1	0	0	1	1	0
0	1	1	0	0	0	0	0
1	1	1	0	0	1	1	1
0	1	1	1	1	0	1	0
1	1	1	1	1	0	1	1

Table 1: State Transition Table with Flip-Flop Inputs

3.5 Karnaugh Maps and Boolean Expression Derivations

Let the current state bits be represented by Q_2, Q_1, Q_0 . The next-state bits for the D flip-flops are defined as D_2, D_1 , and D_0 . The Karnaugh maps are created for each D input to derive simplified Boolean expressions.

3.5.1 Karnaugh Map for D_0

		Q_1Q_0			
		00	01	11	10
$In\ Q_2$	00	0	0	0	0
	01	X	X	0	0
	11	X	X	1	1
	10	1	1	1	0

$$D_0 = (In)(\overline{Q_1} + Q_0 + Q_2)$$

3.5.2 Karnaugh Map for D_1

		Q_1Q_0			
		00	01	11	10
$In \ Q_2$	00	0	0	1	0
	01	X	X	1	0
	11	X	X	1	1
	10	0	1	1	1

The simplified Boolean expression derived from this K-map is:

$$D_1 = Q_0(In + Q_1) + Q_1(In)$$

3.5.3 Karnaugh Map for D_2

		Q_1Q_0			
		00	01	11	10
$In \ Q_2$	00	0	0	0	0
	01	X	X	0	0
	11	X	X	0	1
	10	0	0	0	1

The simplified Boolean expression for D_2 is:

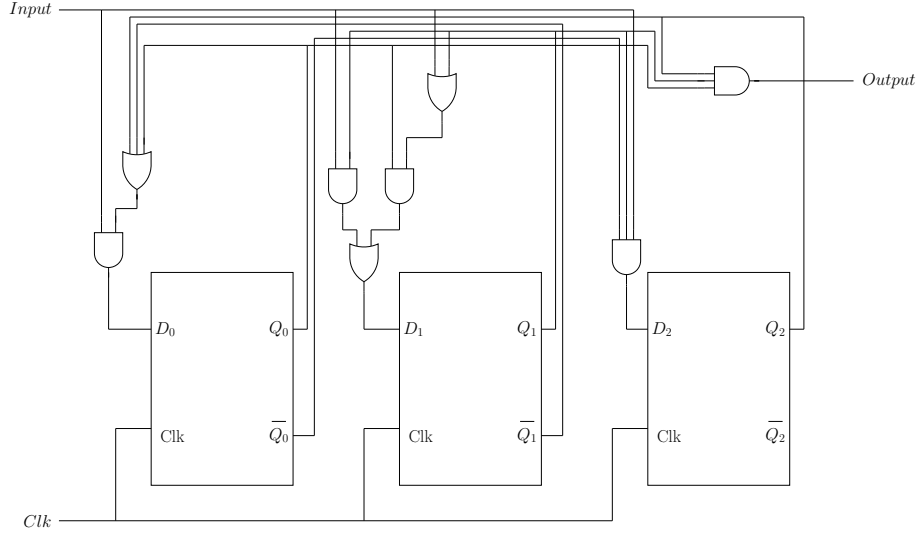
$$D_2 = (In)Q_1Q_0$$

3.5.4 Boolean expression for Op

Op will only be 1 in S_5 ,

$$Op = Q_2Q_1Q_0$$

3.6 Circuit Implementation and Diagram



The sequence detector circuit is implemented using:

- 2 7474 D flip-flops IC (3 D flipflops) in for state storage
- Logic gates (AND, OR) to implement the simplified Boolean expressions
- An Arduino to generate the input sequence
- An LED to display the output
- An oscilloscope for monitoring the waveform across the circuit

4 Procedure

1. Set up the Arduino Uno to generate the test sequence and clock signal.
2. Wire the 7474 D flip-flops according to the state encoding and next-state logic.
3. Implement the combinational logic for the next-state functions using AND, OR, and NOT gates.
4. Connect the output function to an LED through an appropriate resistor.
5. Connect the input, and output signals to the oscilloscope for monitoring.

4.1 Arduino Code

```
const int dataOutPin = 2;      // Pin to output bit sequence
const int clkPin = 3;         // Clock output pin
const int inputPin = 4;       // Input pin to read from
const int clockDelay = 500;   // Clock period in ms

String bitSequence = "110";   // Repeating bit sequence
int bitIndex = 0;

void setup() {
  pinMode(dataOutPin, OUTPUT);
  pinMode(clkPin, OUTPUT);
  pinMode(inputPin, INPUT);

  Serial.begin(9600);
}

void loop() {
  // Output bit from the sequence
  int bitValue = bitSequence[bitIndex] - '0';
  digitalWrite(dataOutPin, bitValue);
  delay(10);

  // Clock rising edge
  digitalWrite(clkPin, HIGH);
  delay(clockDelay);

  // Read and print input pin value
  Serial.println(inputValue);

  // Falling edge of clock
  digitalWrite(clkPin, LOW);
  delay(clockDelay);

  // Move to next bit in the sequence (loop around)
  bitIndex = (bitIndex + 1) % bitSequence.length();
}
```

5 Observations

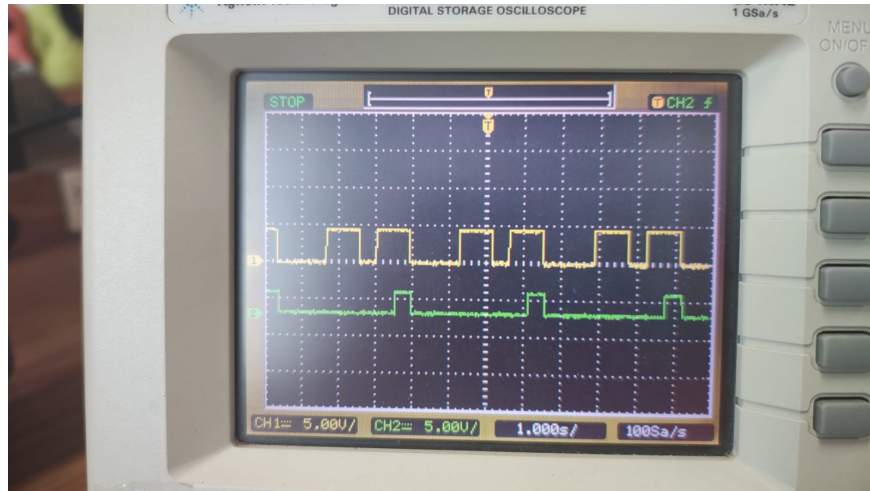


Figure 1: Oscilloscope reading for 00011011 repeated sequence

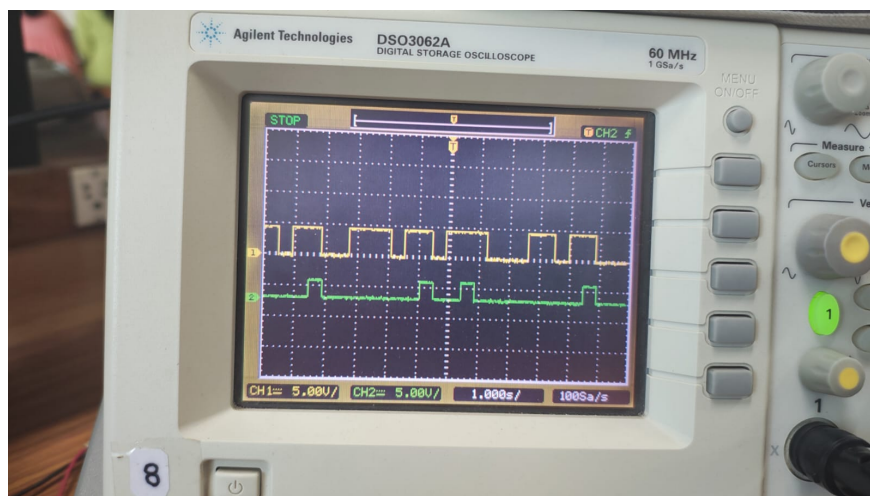


Figure 2: Oscilloscope reading for 0001101100111011011100011011 repeated sequence

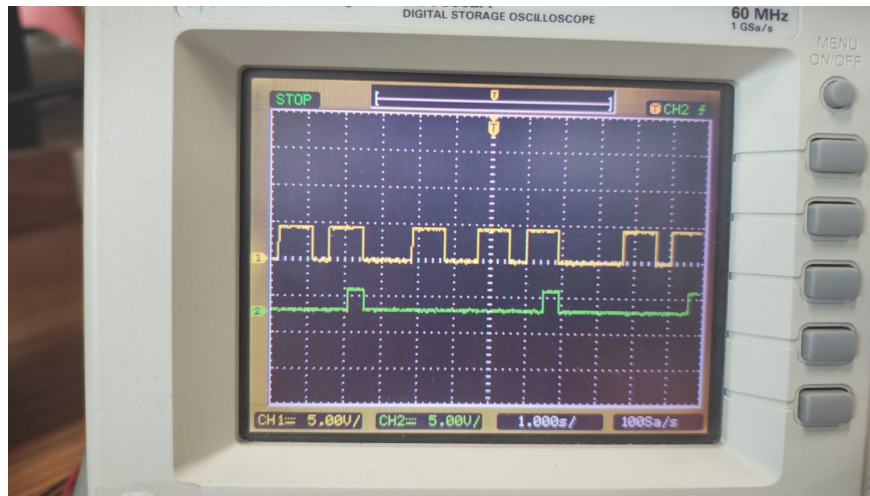


Figure 3: Oscilloscope reading for 001101100001101100011 repeated sequence

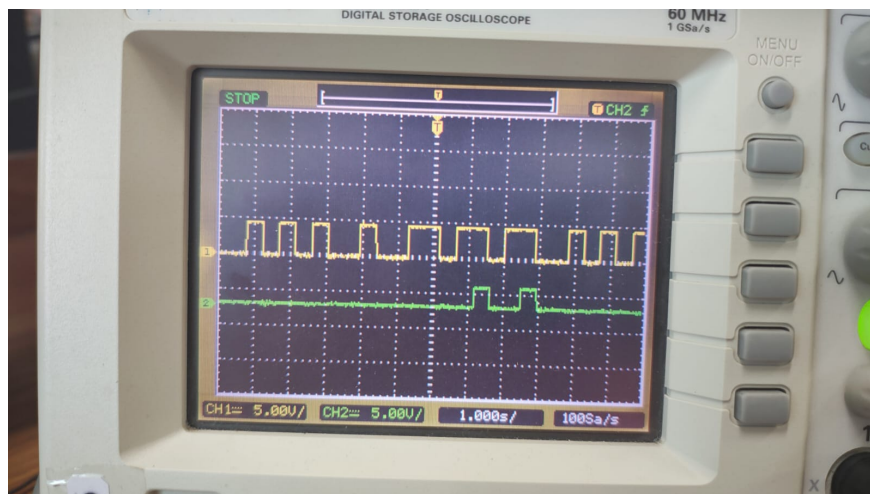


Figure 4: Oscilloscope reading for 00110110110010101001 repeated sequence

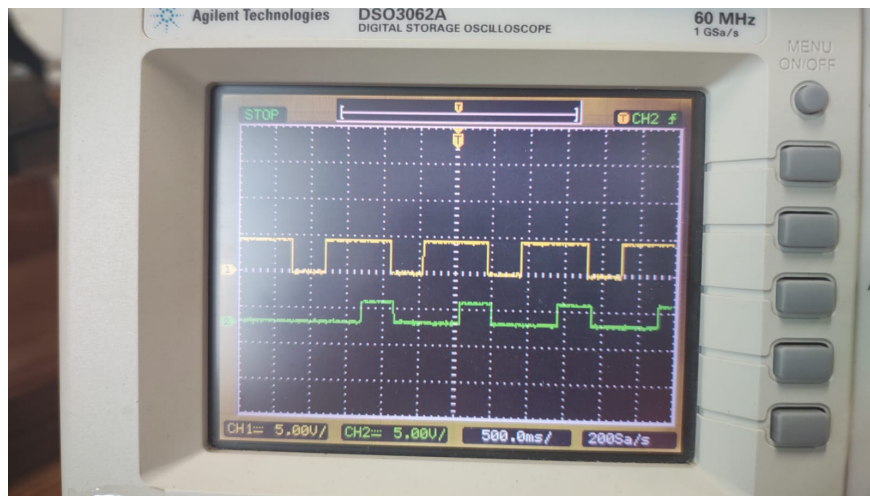


Figure 5: Oscilloscope reading for 011011 repeated sequence

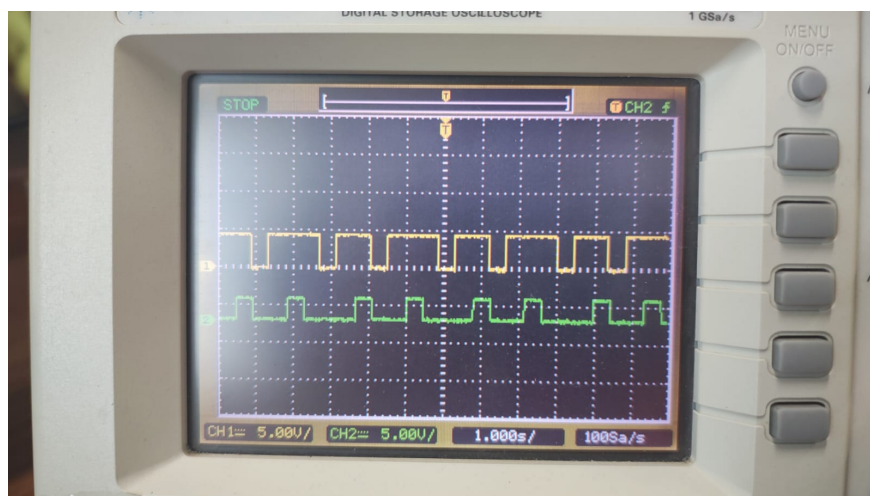


Figure 6: Oscilloscope reading for 1011011 repeated sequence

6 Circuit Picture and Video of Working

Video can be downloaded at [here](#).

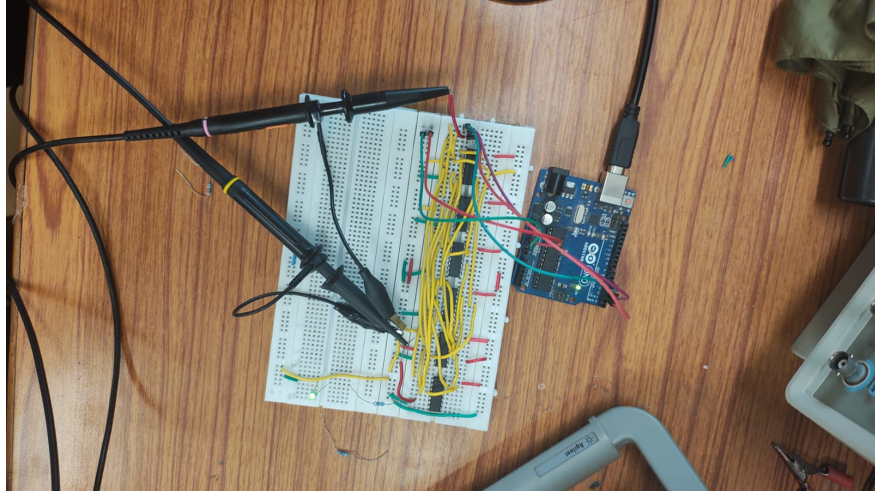


Figure 7: Circuit Picture

7 Conclusion

The Moore machine-based sequence detector for the "11011" pattern was successfully designed and implemented using D flip-flops (7474 ICs). The circuit correctly identified all occurrences of the target sequence in the input stream, including overlapping sequences.